ONNX-Predict: Predicting the Latency of ONNX Models Using Static Features

Siddharth Narsipur
snarsipu@u.rochester.edu
Department of Computer Science, University of Rochester
Rochester, NY, USA

Henry Liu
hliu68@u.rochester.edu
Department of Computer Science, University of Rochester
Rochester, NY, USA

Abstract

ONNX Runtime is the most popular inference engine for machine learning models in the ONNX format. This project studies whether the inference latency of an ONNX model can be predicted using static information that can be obtained ahead of time: data from the ONNX graph and basic hardware features of the target machine. Our results show that static features are informative enough to support useful latency prediction, and our two-tower neural network successfully predicts 87% of models within 10% of actual latency.

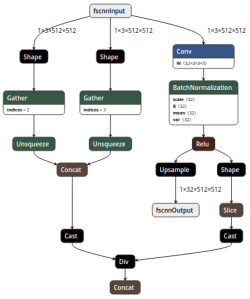

Figure 1: An example of an ONNX model graph.

1 Introduction

Inference latency is an important practical concern in machine learning deployment. Before choosing a model for production, developers often want to know how fast that model will run on a particular machine. This matters for system responsiveness, hardware planning, cost, and model selection. In many cases, the usual solution is repeated benchmarking: run the model on the target hardware and record its latency across a variety of inputs. However, this is not always convenient or even feasible. A target machine may not be available to the developer, testing many variants of candidate models may take too long, and benchmarking every model-hardware combination can become prohibitively expensive.

This project aims to explore an alternative approach: predicting inference latency using only static information. Instead of running a model to measure its speed, the goal is to inspect the model graph, capture the workload it represents, combine that information with hardware specifications, and estimate inference latency. This is useful not only for forecasting performance on unseen machines, but also for understanding which static properties of an ONNX model play the largest role in predicting runtime behavior.

The problem lies in thinking about interaction. A model is not fast or slow by itself, and a CPU is not fast or slow in isolation. What matters is how a particular workload behaves on a particular machine. A model with heavy matrix multiplication may run efficiently on one processor and less efficiently on another. A model that moves large amounts of tensor data may be especially sensitive to cache sizes or memory bandwidth. For that reason, latency prediction cannot be framed only as a property of model size or hardware strength. It is a combination of both model and machine.

The project has three main goals. First, it examines how ONNX Runtime optimization levels relate to model performance. Second, it studies whether inference latency can be predicted for a target machine without direct benchmarking. Third, it investigates which static features are most useful for this prediction task.

Contributions. This work makes the following contributions:

(1) ONNX-Predict, a Python-based tool that can predict the inference latency of any ONNX model across four optimization variants.
(2) We release a large dataset, comprising 143,000 data points, of ONNX Runtime inference data across different hardware.
(3) An evaluation of different machine learning approaches for predicting ONNX model inference latency.

2 Background

2.1 ONNX and ONNX Runtime

ONNX is a standard format for representing machine learning models as computation graphs [7]. In an ONNX graph, each node represents an operation and each edge represents the flow of tensor data between operations. This graph structure makes ONNX a good target for static analysis. Without executing the model, it is possible to inspect which operators appear, how often they appear, and what tensor shapes and data types are involved.

ONNX Runtime is an inference engine designed to execute ONNX models efficiently across multiple hardware backends [1]. ONNX Runtime can also apply graph optimizations before execution. These optimization levels can be understood analogously to compiler optimization levels: the meaning of the computation should remain the same, but the internal graph may be transformed, simplified, or fused in ways that change runtime behavior. However, previous work has found that enabling all or certain optimizations may actually hurt model performance [2, 5]. Due to this, optimization level becomes part of the latency prediction problem rather than a minor implementation detail.

Table 1: ONNX operators considered for operator counts.

Category	Count	Examples
conv_ops	3	Conv, QLinearConv, ConvTranspose
matmul_ops	8	MatMul, Gemm
elementwise_ops	28	Relu, Sigmoid, Tanh, Mul, Div
reduction_ops	15	MaxPool, AveragePool
normalization_ops	6	Softmax, BatchNormalization
movement_ops	19	Transpose, Reshape
other	48	Control flow, constants, shape transformations, boolean logic
total	**127**	

Table 2: Data movement features used in the model.

Feature Group	Features
Compute-bound operations	conv_flops
	matmul_flops
Memory-bound operations	elementwise_mb
	reduction_mb
	movement_mb
	normalization_mb

3 Feature Selection

The static features used in this project fall into two broad groups: graph features and hardware features.

3.1 Graph Features

Graph features describe the structure and the type of workload of the ONNX graph. These features are further divided into operator counts and data movement.

Operator counts are the count of how many times operations like `Conv`, `MatMul`, `Relu`, or `Transpose` appear in the model graph. These counts help distinguish different types of workloads and provide an important signal about the arithmetic intensity the model requires for inference. In all, 127 different types of nodes were considered, as shown in Table 1.

Next, six data movement features were used to estimate how much data each model moves and the amount of floating-point arithmetic it requires. Two of these, `conv_flops` and `matmul_flops`, estimate arithmetic work for convolution-style and matrix-multiplication-style operations. The remaining four summarize memory-heavy behavior by estimating tensor movement associated with elementwise operations, reduction operations, normalization operations, and explicit movement-style operations such as reshaping and transposition. These were calculated using the sizes of the input and output tensors for each operator in the model graph. All the features are listed in Table 2.

3.2 Hardware Features

Hardware features describe the capability of the machine on which inference is performed. These include quantities such as number of cores, memory size, cache sizes, clock speed, memory bandwidth, and CPU type. These features matter because inference latency depends not only on what work the graph implies, but also on how well the hardware can support that work.

Table 3: Hardware features used in the model.

Feature	Description
memory_mb	System memory size in MB
l1d_cache_kb	L1 data cache size in KB
l1i_cache_kb	L1 instruction cache size in KB
l2_cache_kb	L2 cache size in KB
base_clock_mhz	CPU base clock frequency in MHz
num_cores	Number of CPU cores
memory_bandwidth_gbs	Memory bandwidth in GB/s
cpu_provider	CPU vendor, e.g., AMD or Intel

Table 4: Hardware configurations used for inference measurements.

Platform	Cores	Memory
Bluehive		
	1	16 GB
	4	16 GB
	8	32 GB
	8	16 GB
	4	8 GB
	2	32 GB
	4	32 GB
	2	16 GB
Google Cloud		
	2	16 GB
	1	8 GB
	1	16 GB
	4	32 GB
	2	32 GB
	4	16 GB
	2	8 GB
	6	32 GB

4 Dataset Construction

4.1 Data Source

ONNX Model Zoo is the largest open-source collection of ONNX models [8]. The dataset contains 2,326 models from various domains such as computer vision, transformers, classification, labeling, and more.

4.2 Collecting Graph Features

Using the official Hugging Face API, we downloaded all models in the Model Zoo using a Python script. Each model was then converted into four ONNX Runtime optimization variants (enable_all, disable_all, enable_extended, enable_basic). All models were also converted to ONNX opset 21 to eliminate any performance ambiguity due to opset variation.

A separate Python script then collected data on node counts for each operator listed in Table 1. Then, data movement features were

Table 5: Statistics of the inference data captured.

Statistic	Latency (ms)
Mean	201.87
Median	63.36
Standard deviation	592.07
Minimum	0.03
Maximum	43,674.17
P10	11.01
P25	25.81
P75	156.53
P90	399.78
P95	757.79
P99	2,555.19

computed using the tensor data of each model graph. At the end of this stage, our script produced 9,064 ONNX models.

4.3 Inference Measurement

Inference data was collected on Linux x86 machines across two clusters: BlueHive, the University of Rochester's computing cluster, and Google Cloud VMs. For each of the 9,064 models generated above, sixteen timing runs were performed using different combinations of core counts and memory, as listed in Table 4.

To maintain consistency across machines, hyper-threading was disabled. An environment variable was used to control the number of cores available to ONNX Runtime. Data was captured using a Python script that loads each model and generates a random input tensor based on the model's input shape. Inference was done using the ONNX Runtime CPU Provider. Each capture for a model had five warm-up runs, followed by ten inferences that were timed and considered in the final dataset. At the end of this stage, our dataset contained 143,218 data points. Each row had 142 features, comprising 127 node count features, 6 data movement features, 9 hardware features, and the inference data.

As shown in Table 5, the inference latency measurements show substantial variance, with a large standard deviation of 592.07 ms. The distribution is highly right-skewed and heavy-tailed: the fastest model had an average latency of 0.03 ms, while the slowest model took nearly 43 seconds.

5 Experiment Setup

5.1 Data Pre-processing

As some of the timing data was collected on shared machines, some measurements were noisy. To reduce the effect of unstable latency readings, rows with coefficient of variation greater than or equal to 10% were removed. This filtered out 6,382 rows, or about 4.5% of the data. Some numerical features were rounded to reduce unnecessary precision. The CPU provider was encoded as a binary feature, and standardized scaling was applied for all numerical features.

After filtering, the data was split into training, validation, and test sets using a 70/15/15 split. Broad-range numerical features were log-scaled so that extremely large values and outliers would not dominate the learning process.

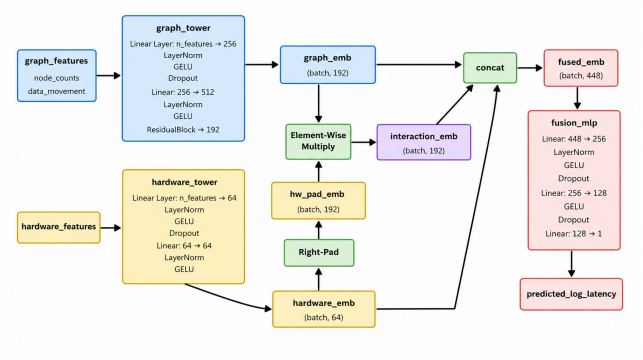

Figure 2: Architecture of two-tower neural network.

5.2 Log-Latency as the Prediction Target

Our models were trained to predict latency in log space rather than directly in milliseconds. This choice reflects the fact that relative error is more meaningful than absolute error for runtime prediction. For example, predicting 2 ms for a true latency of 1 ms is much worse than predicting 1001 ms for a true latency of 1000 ms, even though both predictions have an absolute error of 1 ms. Predicting log-latency therefore better aligns with our objective.

5.3 Prediction Models

Three kinds of models were explored: XGBoost, RandomForestRegressor, and neural networks. The XGBoost and RandomForestRegressor models were trained using their standard implementations, with hyperparameter tuning performed on the validation set.

5.4 Neural Network Architecture

Our neural network used a two-tower architecture that separated the feature space into graph-side and hardware-side inputs. The two-tower model is a machine learning architecture created for use in recommendation systems [9]. It is particularly useful in finding a relationship between two different types of data, such as matching a text query to an image, or in our case, the relationship between graph features and hardware.

The model architecture is shown in Figure 2. The model first processes graph features through a graph tower and hardware features through a separate hardware tower, producing graph and hardware embeddings, respectively. The hardware embedding is then right-padded with zeros to match the dimensionality of the graph embedding. These embeddings are element-wise multiplied to produce an interaction embedding. Finally, the graph embedding, hardware embedding, and interaction embedding are concatenated and passed to a fusion MLP, which outputs the predicted log latency.

Since latency depends not only on the model or hardware independently but also on how well a particular workload maps onto a particular machine, the interaction embedding helps capture workload-hardware compatibility. For example, a compute-heavy graph may benefit more from higher core counts or clock speed, while a memory-heavy graph may be more sensitive to memory bandwidth and cache size. Using all three embeddings in the final MLP gives it access to both independent effects and cross-effects,

Table 6: Overall test-set performance across model types.

Model Type	Median Rel Err	p90 Rel Err	Median % Err	p90 % Err	Median Ratio	Within 10%	Within 25%
XGBoost	0.082	0.420	8.23%	42.06%	1.08	55.8%	80.8%
RandomForest	0.020	0.207	2.06%	20.73%	1.02	79.6%	92.1%
Neural Network	0.023	0.132	2.00%	13.30%	1.02	87.3%	94.2%
Random Guess (Mean)	–	–	218.59%	–	–	3.91%	9.46%
Random Guess (Median)	–	–	74.25%	–	–	6.57%	15.12%

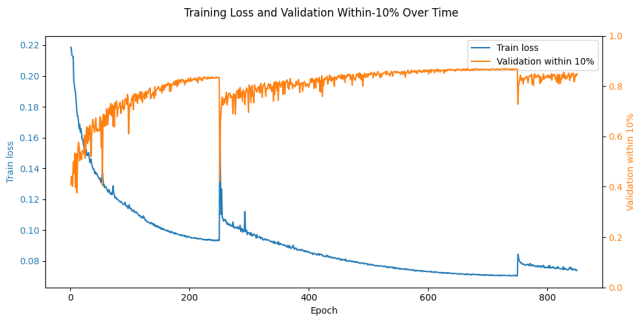

Figure 3: Training loss and within-10% validation accuracy for the best-performing neural network training run.

allowing it to learn more flexible predictions across different models and hardware configurations. The graph tower answers the question "what kind of workload is this?" while the hardware tower answers "what kind of machine is this?" The final prediction depends on how those two answers interact.

5.5 Model Training

The neural network was trained on Google Colab using an NVIDIA G4 GPU with PyTorch. Training ran for 850 epochs with a batch size of 1024, taking approximately 20 minutes in total, with the best checkpoint saved at epoch 666. The optimizer was AdamW with a learning rate decayed from 8e-4 to 2e-4 via CosineAnnealingWarmRestarts, restarting every 250 epochs. SmoothL1Loss was used as the training loss.

6 Results

6.1 Evaluation

Performance was evaluated mainly through relative-error-based metrics, since these are more appropriate than raw absolute error for latency prediction. In this setting, a small absolute mistake on a very fast model can be much more serious than the same absolute mistake on a slow model.

Table 6 summarizes the main results. Among the three models, the neural network performed best overall. It achieved the lowest median relative error at 0.023, which corresponds to a median percentage error of 2.00%, and it also had the lowest 90th-percentile relative error at 0.132. This means the neural network not only gave the best typical predictions, but also remained more reliable on harder cases. It placed 87.3% of predictions within 10% error and 94.2% within 25% error.

The Random Forest model also performed strongly. Its median relative error was 0.020, very close to the neural network, and 79.6% of its predictions were within 10% error. However, its p90 relative error of 0.207 was noticeably worse than the neural network's, suggesting that it was less stable on more difficult examples even though its typical-case performance remained good.

XGBoost was the weakest of the three trained models. Its median relative error was 0.082 and its p90 relative error was 0.420, with 55.8% of predictions within 10% error and 80.8% within 25% error. This indicates that XGBoost captured useful structure in the data, but did not model the full range of workload-hardware interactions as effectively as the other approaches.

Overall, these results show that the prediction task is very feasible, but model choice matters. The neural network gave the best balance of typical accuracy, tail-case robustness, and strict accuracy thresholds, while Random Forest remained a simpler, strong alternative. Both substantially outperformed random baselines, supporting the conclusion that static features contain enough information to make useful latency predictions.

6.2 Feature Selection

We initially trained the models using only subsets of the collected features, such as the top 10 operator counts or only the data-movement features in the graph tower. However, all three models achieved better performance when provided with the full feature set. Operator-count features made up most of the feature space and were also among the most informative features. Although these features were sparse, removing them led to a noticeable drop in performance (10-15% on within-10% accuracy). This indicates that the operator composition of an ONNX graph contains useful information about inference behavior that cannot be fully captured by broader aggregate features alone.

We also explored additional hand-engineered features, but these did not consistently help. Some metrics improved while others worsened, suggesting that the extra features added noise or bias rather than stable predictive value.

6.3 Differences Across Hardware Groups

Prediction quality was not equally strong across all hardware subsets. Accuracy was better for some core-count settings than others, and the miss rate was noticeably higher on AMD machines than on Intel machines, as shown in Tables 8 and 7. This suggests that some machine families were harder to model accurately than others with our subset of hardware features.

Table 7: Prediction performance by platform and CPU provider.

Platform	CPU Provider	Within 10%	Within 25%
Bluehive	Intel	90.9%	93.7%
Google Cloud	AMD	82.7%	94.9%
Google Cloud	Intel	89.4%	93.0%

Table 8: Prediction performance by CPU core count.

Cores	Within 10%	Within 25%	Within 50%
1	91.8%	96.4%	98.7%
2	85.6%	94.7%	98.0%
4	85.0%	91.9%	95.5%
6	91.0%	93.0%	95.2%
8	90.8%	93.3%	94.9%

6.4 CLI Tool

We also designed a CLI-based tool built on the trained neural network [6]. Given an ONNX model, the tool predicts its expected inference latency and reports a confidence label based on the model's historical accuracy for similar model-hardware combinations. We also released both the neural network model [4] and the dataset [3] on Hugging Face.

7 Limitations

This study was intentionally narrow in scope. It only considers ONNX Runtime on Linux x86 CPUs, so the results do not automatically extend to GPUs, other operating systems, or other inference runtimes.

The feature set is also based on static approximations rather than direct runtime measurements. Operator counts, FLOP estimates, and memory-movement estimates capture useful structure, but they do not fully represent effects such as fusion quality, cache reuse, thread scheduling, or kernel implementation details.

Finally, the dataset still has limits in both measurement quality and coverage. The timing data was collected on shared machines using synthetic inputs, so some noise remains even after filtering. Additionally, generalization is likely to be strongest when new models and hardware resemble those seen during training.

8 Future Work

The natural next step is to extend the approach beyond CPU inference, especially to GPU settings. That would make the predictor more broadly useful. Another direction is to improve hardware coverage. The weaker results on some hardware groups suggest that collecting data from a wider range of systems would likely improve robustness and generalization.

9 Conclusion

This project shows that ONNX inference latency can be predicted from static information alone. By combining graph-level operator counts, workload estimates, and hardware descriptors, we show that it is possible to learn a useful mapping from model structure and machine characteristics to expected runtime. By using a two-tower neural architecture, we explicitly model the interaction between workload characteristics and hardware capabilities, allowing the network to capture not just how complex a model is, but how well it maps onto a particular machine. Our best model places 87% of predictions within 10% of actual latency, substantially outperforming both simple baselines and tree-based approaches on tail cases.

10 Acknowledgments

The authors thank the Center for Integrated Research Computing (CIRC) at the University of Rochester for providing computational resources. We also thank Professor Sreepathi Pai, Professor Zhiyao Duan, and Jingyu Qiu for assistance and advice throughout the project.

References

[1] ONNX Runtime developers. 2021. ONNX Runtime. https://onnxruntime.ai/. Version: x.y.z.

[2] Nikolaos Louloudakis and Ajitha Rajan. 2025. OODTE: A Differential Testing Engine for the ONNX Optimizer. https://arxiv.org/abs/2505.01892. Preprint — arXiv:2505.01892; accessed 08Dec2025.

[3] Siddharth Narsipur. 2026. ONNX Inference Dataset. https://huggingface.co/datasets/sidnarsipur/onnx-inference. Accessed: 2026-05-08.

[4] Siddharth Narsipur. 2026. ONNX-Predict: Trained Model for ONNX Inference Latency Prediction. https://huggingface.co/sidnarsipur/onnx-predict. Accessed: 2026-05-08.

[5] Siddharth Narsipur. 2026. OTX: Testing the ONNX Runtime Optimizer. https://github.com/sidnarsipur/OTX. GitHub repository.

[6] Sid Narsipur and Henry Liu. 2026. ONNX-Predict: Predicting the Latency of ONNX Models Using Static Features. https://github.com/sidnarsipur/onnx-predict. GitHub repository.

[7] ONNX Community. 2024. Open Neural Network Exchange (ONNX). https://onnx.ai/. Accessed: 8 May 2026.

[8] onnxmodelzoo / Hugging Face. [n. d.]. onnxmodelzoo — ONNX Model Zoo on Hugging Face. https://huggingface.co/onnxmodelzoo. Accessed: 2025-12-08.

[9] Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed H Chi. 2019. Sampling-bias-corrected neural modeling for large corpus item recommendations. In *Proceedings of the 13th ACM Conference on Recommender Systems*. 269–277. https://dl.acm.org/doi/10.1145/3298689.3346996